

ThinQ Workspace Sentinel · 팀 온보딩 가이드 v2

LG DX School 5기 · 5분 대기조 · 2026.05.19 ~ 06.25 · 발행 2026-06-01 · v2: 로컬 + GitHub PR 워크플로 채택

0. 한 줄 소개 · 일하는 방식

요양병원 RSV·인플루엔자·노로를 3주 전에 예측해서 LG ThinQ 가전 8종이 자동 환경 제어하는 시스템.

[v2 핵심 변경]

팀원은 본인 PC에서 코드 작성 (VM 접속 X)

docker compose 한 줄로 로컬 mini-stack 실행 (DB·Redis·API)

feature/* → PR → 자동 CI → develop → main → 자동 VM 배포

VM은 통합·시연 전용 (박진영 운영)

UIS와의 관계: ~/urban-immune-system/ 는 다른 캡스톤 팀이 운영하는 외부 데이터 공급자. KOWAS·DataLab·OTC 정제 데이터만 read-only로 받음. UIS 디렉토리·UIS 스키마(public.*) 절대 수정 금지.

1. 사전 준비 (본인 PC)

Git · GitHub 계정	박진영에게 GitHub ID 전달 → Collaborator 등록
Docker Desktop	https://www.docker.com/products/docker-desktop/ — 필수
Node 20+	Frontend 작업자 · nvm install 20
Python 3.11	Backend·ML·Pipeline 작업자 · pyenv 권장
IDE	자유 (VSCode·JetBrains 등)

2. 첫날 셋업 (15분)

```
# 1) 클론
git clone https://github.com/zln02/thinq-workspace-sentinel.git
cd thinq-workspace-sentinel

# 2) develop 동기화 + 본인 feature 브랜치
git checkout develop && git pull
git checkout -b feature/〈모듈〉-〈기능〉-w3

# 3) 로컬 mini-stack 실행 (Docker Desktop 켜져 있어야 함)
docker compose -f infra/docker-compose.dev.yml up -d

# 4) 헬스체크
curl http://127.0.0.1:8003/health

# 5) 시뮬레이션 실행
curl -X POST http://127.0.0.1:8003/api/v1/simulate \
-H "Content-Type: application/json" \
-d '{"scenario":"summer_norovirus","minutes":60}'

# 6) 종료
docker compose -f infra/docker-compose.dev.yml down
```

3. 역할 분담 (5분 대기조)

※ 이름란 〈 〉는 박진영(Tech Lead)이 실제 멤버 이름으로 채울 것.

모듈	담당	W3~W6 작업
ml/	박진영 Tech Lead · ML	XGBoost+TFT, REHVA, 5-Tier, 전체 아키텍처
backend/	〈Backend 담당〉	FastAPI 확장, Smart Protocol, SSE 실시간 게이트웨이

pipeline/	〈Data 담당〉	UIS DB 어댑터(read-only), Kafka 구독, 시뮬 시나리오
frontend/	〈Frontend 담당〉	Next.js 14 · /admin · /mobile · /demo 3종 클라이언트
infra/	〈DevOps/QA 담당〉	GitHub Actions 유지, VM 모니터링, 배포 자동화

4. Git 워크플로 (반드시 준수)

feature/<모듈>-<기능> — PR — develop — PR — main — auto — VM 배포

브랜치 네이밍

좋은 예	나쁜 예
feature/backend-sse-realtime-w3	dev (모호)
feature/frontend-admin-grid-w3	김나영-작업 (이름 X)
feature/pipeline-uis-adapter-w3	temp
fix/migration-002-rollback	master (사용 안 함)

절대 금지

main · develop 직접 push
.env · 비밀키 commit (CI가 차단)
--no-verify 옵션 (pre-commit 우회)
UIS 디렉토리(~/urban-immune-system/) · UIS 스키마(public.*) 수정
본인 모듈 외 직접 수정 (PR로만)

PR 규칙

PR 템플릿 자동 로드 (어떤 모듈·왜·검증·UIS 경계 4섹션)
최소 1명 리뷰 필수 (Tech Lead 또는 다른 모듈 담당)
CI 4개 (lint·smoke·frontend·secret-scan) 모두 통과해야 머지
PR 리뷰 SLA 24h 내 1차 응답

5. 자동 배포 흐름 (CD)

```
GitHub —[push to main]—> GitHub Actions —[SSH]—> VM (uis-capstone)
|
├ git fetch → main reset --hard
├ migrations 적용 (있으면)
├ sentinel-api 재시작
└ /health smoke check
```

자동 트리거: main 머지 시

수동 트리거: Actions 탭 → "Deploy to VM" → Run workflow (롤백 등)

배포 실패 시 박진영 즉시 확인

VM Secrets (박진영 설정): VM_SSH_KEY · VM_HOST · VM_USER

6. 주차별 마일스톤

주차	공동 목표	모듈별 핵심
W3 6/2~8	외부신호 → 예측 → 시물 통합	pipeline: UIS DB 어댑터 / ml: XGBoost / backend: SSE
W4 6/9~15	Smart Protocol + 75초 시연 영상	frontend: 평면도+그리드+demo / backend: 정책
W5 6/16~22	다층 게이트·자동 해제·모바일	ml: FAR / frontend: PWA / infra: 모니터링
W6 6/23~25	최종 발표·문서·시연 영상	전체 발표·PPT·라이브 리허설

7. 디버깅 첫 5가지

1. Docker Desktop 안 켜짐

켜고 docker ps 확인

2. 포트 충돌

로컬 dev 포트 (55432 · 56379 · 8003) 다른 앱이 쓰는지 확인

3. DB 마이그레이션 재적용

```
docker compose -f infra/docker-compose.dev.yml down -v && up -d
```

4. CI 실패

PR Actions 탭 로그 확인 → 로컬에서 `ruff check backend pipeline` 재현

5. 자동 배포 실패

박진영에게 즉시 알림 → Actions 탭에서 SSH 출력 확인

8. 소통

Slack 채널: #sentinel-dev (박진영 개설)

데일리 스탠드업: 매일 21:00 KST · 10분 · 모듈별 1줄

블로커: 24h 이상 막히면 박진영에게 DM

PR 리뷰 SLA: 24h 내 1차 응답

ThinQ Workspace Sentinel v2 · 박진영 (Tech Lead) · 본 문서 변경은 docs/team_onboarding_guide.md 기준 · GitHub: github.com/zln02/thinq-workspace-sentinel (private)